KSBL — Object-Oriented Programming

FINAL PROJECT REPORT

PayrollOS

Payroll Management System

Group Member	Student ID
M. Ammar Saleem	B04-0925-000130
Fatima Naeem	B04-0925-000108
Abdul Hadi	B04-0925-000104

Spring 2026 · v1.0

1. Executive Summary

PayrollOS is a desktop application built in Java that automates the complete lifecycle of employee salary processing — from secure login to payslip generation to real-time salary analytics. The system serves three distinct roles: Employee, Sub Admin, and Super Admin, each with a tailored dashboard and strictly enforced permissions.

The frontend is built with JavaFX (FXML + Controllers) in a clean MVC architecture, while the backend consists of pure Java classes covering authentication, payroll calculation, attendance, leave management, and CSV-based persistence. The system ships with 28 seed employees across Full-Time, Part-Time, and Contract categories.

2. System Architecture

2.1 MVC Layer Overview

App Layer	AppLauncher (JavaFX entry), AppData (global state), DataStore (CSV persistence), Main (data init & seeding)
Auth Layer	UserAccount hierarchy: EmployeeAccount, SubAdminAccount, SuperAdminAccount — each enforces role-specific business methods
Controller Layer	LoginController, RoleSelectController, AdminSelectController, SuperAdminController, SubAdminController, EmployeeController
Model Layer	Employee (abstract), FullTimeEmployee, PartTimeEmployee, ContractEmployee, PaySlip, PayrollManager, LeaveRequest, AttendanceRecord, Allowance, Deduction
Reporting Layer	ReportEngine — console-based reports; UI controllers render the same data in JavaFX TableViews and VBoxes

2.2 Project Structure

Package / Path	Contents
app/	AppLauncher, AppData, DataStore, Main
auth/	UserAccount, EmployeeAccount, SubAdminAccount, SuperAdminAccount
controller/	LoginController, RoleSelectController, AdminSelectController, SuperAdminController, SubAdminController, EmployeeController
model/	Employee (abstract), FullTimeEmployee, PartTimeEmployee, ContractEmployee, PaySlip, PayrollManager, LeaveRequest, AttendanceRecord, Allowance, Deduction
enums/	EmployeeRole, AccessRole, AllowanceType, DeductionType, AttendanceStatus, LeaveStatus
interfaces/	IPayable, ISalaryCalc
reporting/	ReportEngine
resources/fxml/	roleselect.fxml, adminselect.fxml, login.fxml, superadmin.fxml, subadmin.fxml, employee.fxml
employees.csv	Auto-created on first run; persists all employee + account data across restarts

3. Data Model & Class Hierarchy

3.1 Employee Hierarchy

Class	Salary Formula	Key Fields	Validation
FullTimeEmployee	baseSalary + allowances + (2000 × overtimeHrs)	allowances (double), overtimeHrs (int)	allowances >= 0, overtimeHrs >= 0
PartTimeEmployee	hourlyRate x hoursWorked	hourlyRate (double), hoursWorked (int)	hourlyRate > 0, hoursWorked >= 0
ContractEmployee	contractRate (flat monthly)	contractRate (double), contractEnd (LocalDate)	contractRate > 0, contractEnd != null

Employee is abstract and implements IPayable (computeNetSalary) and ISalaryCalc (calculateSalary). Every subclass overrides calculateSalary(). Net salary is computed in the parent by calling calculateSalary() and subtracting all Deduction amounts. Internal lists are exposed only as unmodifiable views to enforce encapsulation.

3.2 Enumerations

Enum	Values	Used In
EmployeeRole	FULL_TIME, PART_TIME, CONTRACT	Employee subclasses, DataStore CSV parsing, Add Employee form
AccessRole	SUPER_ADMIN, SUB_ADMIN, EMPLOYEE	All UserAccount subclasses, login routing
AllowanceType	HOUSING, TRANSPORT, MEDICAL, BONUS	Allowance model, My Report display
DeductionType	TAX, PENSION, INSURANCE, FINES	Deduction model, My Report display
AttendanceStatus	PRESENT, LATE, ABSENT, LEAVE	AttendanceRecord, attendance TableViews
LeaveStatus	PENDING, APPROVED, REJECTED	LeaveRequest, leave management tables and history

4. Authentication & Role-Based Access

Class	Role	Unique Capabilities
SuperAdminAccount	SUPER_ADMIN	View all employees, approve/reject leaves, create Sub Admin accounts at runtime via promoteToSubAdmin(), manage all employees
SubAdminAccount	SUB_ADMIN	Manage payroll, view all reports, manage attendance, approve and reject leaves (canPromote = false)
EmployeeAccount	EMPLOYEE	View own payslip, view own attendance, submit leave requests, generate own report, use salary calculator

5. Persistence — DataStore & CSV

DataStore is a static utility class that reads and writes all employee and login data to employees.csv. There is no external database — the CSV acts as a lightweight flat-file store. Leave requests and attendance records are runtime-only and are not persisted in this version.

Employee Type	CSV Column Layout
FULL_TIME	id, name, age, dept, FULL_TIME, baseSalary, allowances, overtimeHrs, username, password
PART_TIME	id, name, age, dept, PART_TIME, hourlyRate, hoursWorked, username, password
CONTRACT	id, name, age, dept, CONTRACT, contractRate, contractEnd (YYYY-MM-DD), username, password

6. Payroll Processing Logic

calculateSalary()	Defined in each concrete subclass. Returns gross salary before any deductions.
computeNetSalary()	Defined once in Employee. Calls calculateSalary() then subtracts the sum of all Deduction.getDeduction() values.
processMonthly()	PayrollManager iterates every employee, calls both methods, creates a PaySlip with a unique ID (PS-YYYY-MM-N), appends to payrollHistory.

7. OOP Concepts Applied

Concept	Application in Project
Inheritance	FullTimeEmployee, PartTimeEmployee, ContractEmployee extend abstract Employee; EmployeeAccount, SubAdminAccount, SuperAdminAccount extend UserAccount
Abstraction	Employee.calculateSalary() is abstract; IPayable and ISalaryCalc define contracts enforced across the hierarchy
Encapsulation	All Employee fields are private; lists returned as unmodifiable views (Collections.unmodifiableList) to prevent external mutation
Polymorphism	PayrollManager.processMonthly() calls employee.calculateSalary() polymorphically on every Employee reference regardless of concrete type
Interfaces	IPayable (computeNetSalary) and ISalaryCalc (calculateSalary) enforce consistent salary contracts across all three employee types
Composition	Employee owns Allowance and Deduction lists — their lifecycle is tied to the Employee instance
Aggregation	Employee holds AttendanceRecord and LeaveRequest references — these exist independently
MVC Pattern	Model classes fully decoupled from JavaFX; Controllers mediate between FXML views and model/auth logic
Records	DataStore.LoadResult is a Java record — immutable container for employees + accounts returned from DataStore.load()
Pattern Matching	DataStore.saveAll() uses pattern-matching instanceof (emp instanceof FullTimeEmployee ft) to serialise type-specific CSV fields

8. Complete Method Reference — Frontend & Backend

Every public and package-level method in the system, organised by class and tagged with its layer (Frontend = JavaFX controller/FXML, Backend = auth/model/reporting/app).

`AppLauncher` — Frontend / App Layer

Method	Category	What It Does
start(primaryStage)	JavaFX	Loads roleselect.fxml as the first scene, sets the window title, and shows the window — the JavaFX application entry point.
main(args)	Entry Point	Calls Main.initData() to build all in-memory data, then calls launch(args) to start the JavaFX thread.

`AppData` — Backend / App Layer

Method	Category	What It Does
AppData(manager, superAdmin, subAdminAccounts, employeeAccounts, allEmployees, allLeaveRequests)	Data Init	Constructor — stores all six global collections so any controller or account class can access them via AppLauncher.data.

`DataStore` — Backend / App Layer

Method	Category	What It Does
saveAll(employees, accounts)	Persistence	Iterates the employee list, matches each to its login account, serialises all fields to a CSV row using pattern-matching instanceof, and writes the file. Called every time an employee is added.
load()	Persistence	Reads employees.csv line-by-line, parses each row by type, reconstructs Employee and EmployeeAccount objects, and returns them in a LoadResult record. Returns empty lists on first run.
escapeCsv(s)	Utility	Wraps a field in quotes and escapes internal quotes if the value contains commas, quotes, or newlines — ensures the CSV stays parseable.

`Main` — Backend / App Layer

Method	Category	What It Does
initData()	Data Init	Attempts to load from CSV first; falls back to seed data on first run. Creates leave requests, runs processMonthly(), injects PayrollManager into accounts, builds admin accounts, and returns a populated AppData.
buildSeedEmployees()	Data Init	Creates all 28 default employees (10 full-time, 10 part-time, 8 contract) and calls addSeedAllowancesDeductions() — used only when no CSV exists.
addSeedAllowancesDeductions(list)	Data Init	Assigns Housing/Transport/Medical/Bonus allowances and Tax/Pension/Insurance deductions to the first 10 full-time employees in the seed dataset.

Method	Category	What It Does
buildSeedAccounts(employees)	Data Init	Creates two EmployeeAccount objects (ali123 and sara456) linked to the first two employees — default login accounts for demo use.

`UserAccount` — Backend / Auth Layer

Method	Category	What It Does
login(enteredUsername, enteredPassword)	Auth	Compares entered credentials against stored values; returns true if both match — base logic reused by all three subclasses.
logout()	Auth	Prints a confirmation message; called when any user clicks Logout in the UI.
getRawPassword()	Auth	Returns the stored plain-text password — used by DataStore.saveAll() to persist credentials to CSV.
getUsername()	Getter	Returns the stored username string.

`EmployeeAccount` — Backend / Auth Layer

Method	Category	What It Does
login(enteredUsername, enteredPassword)	Auth	Delegates to super.login() and prints a success/failure message; called by LoginController during employee sign-in.
setPayrollManager(payrollManager)	Core Payroll	Injects the PayrollManager so this account can retrieve payslips — called in Main.initData() and after adding a new employee.
viewOwnPayslip(month)	Reporting	Looks up the PaySlip for this employee and month via PayrollManager, then prints ID, Gross, and Net — console equivalent of the UI payslip panel.
viewAttendance(month)	Attendance	Filters the employee's attendance records by month and prints each date, status, and hours — console equivalent of the attendance TableView.
generateOwnReport()	Reporting	Prints full employee details, gross salary, and net salary to console — console version of the employee self-report screen.
submitLeaveRequest(requestId, startDate, endDate)	Leave Mgmt	Creates a new PENDING LeaveRequest and adds it to the employee's list — called by the Submit button in EmployeeController.showLeaveForm().
getEmployee()	Getter	Returns the linked Employee object — used by controllers and DataStore to match accounts to employees.

`SubAdminAccount` — Backend / Auth Layer

Method	Category	What It Does
managePayroll()	Core Payroll	Validates that a PayrollManager is set and prints a confirmation — confirms the Sub Admin's payroll management responsibility.

Method	Category	What It Does
viewReports()	Reporting	Instantiates ReportEngine and calls salarySummaryReport() and deptPayrollReport() — console equivalent of the salary stats panel.
manageAttendance(employee, month)	Attendance	Counts attendance records for a given employee and month using a stream filter and prints the total.
approveLeave(request)	Leave Mgmt	Validates the request is not null, sets its status to APPROVED — same logic as the Approve button in SubAdminController.
rejectLeave(request)	Leave Mgmt	Sets a LeaveRequest status to REJECTED — called by the Decline button in SubAdminController.
viewAllLeaveRequests()	Leave Mgmt	Iterates all leave requests and prints ID, employee name, and status — console equivalent of the leave management table.

`SuperAdminAccount` — Backend / Auth Layer

Method	Category	What It Does
promoteToSubAdmin(username, password, manager)	Auth	Creates and returns a new SubAdminAccount linked to the shared leaveRequests list — called by SuperAdminController.showCreateSubAdmin().
viewAllLeaveRequests()	Leave Mgmt	Prints all leave requests — scoped to Super Admin.
approveLeave(request)	Leave Mgmt	Validates and sets LeaveRequest status to APPROVED — called by the Approve button in SuperAdminController.
rejectLeave(request)	Leave Mgmt	Validates and sets LeaveRequest status to REJECTED — Super Admin can also decline requests.
manageAllEmployees()	Core Payroll	Iterates allEmployees and prints getDetails() for each — console equivalent of the employee TableView.
getUsername()	Getter	Overrides UserAccount.getUsername() to return the stored username.

`LoginController` — Frontend / Controller Layer

Method	Category	What It Does
setMode(mode)	Login Flow	Called before the scene is shown; sets the login mode (SUPER_ADMIN / SUB_ADMIN / EMPLOYEE) and updates the hint label text.
initialize(url, rb)	JavaFX	Empty Initializable implementation — mode and UI are configured externally via setMode().
handleLogin()	Login Flow	Reads credentials, switches on mode to check the correct account tier, stores the matched account in AppLauncher, and loads the appropriate dashboard FXML. Shows error label on failure.
goBack()	Navigation	Navigates back to roleselect.fxml (employee mode) or adminselect.fxml (admin modes) when the Back button is clicked.
loadScene(fxml)	Navigation	Private helper that loads any FXML from /fxml/ and swaps the current Stage scene — reused by handleLogin() and goBack().

`RoleSelectController`

Frontend / Controller Layer

Method	Category	What It Does
selectAdmin(e)	Navigation	Loads adminselect.fxml when the Admin button is clicked — routes the user to the admin type selector.
selectEmployee(e)	Navigation	Loads login.fxml and calls ctrl.setMode(EMPLOYEE) so the login screen knows which account tier to authenticate against.

`AdminSelectController`

Frontend / Controller Layer

Method	Category	What It Does
selectSuperAdmin(e)	Navigation	Calls navigate() with Mode.SUPER_ADMIN — routes to the login screen configured for Super Admin credentials.
selectSubAdmin(e)	Navigation	Calls navigate() with Mode.SUB_ADMIN — routes to the login screen configured for Sub Admin credentials.
goBack(e)	Navigation	Loads roleselect.fxml — returns to the initial role selection screen.
navigate(e, mode)	Navigation	Private helper that loads login.fxml, retrieves its controller, sets the mode, and swaps the scene.

`SuperAdminController`

Frontend / Controller Layer

Method	Category	What It Does
initialize(url, rb)	JavaFX	No-op Initializable — content area starts empty and is populated on sidebar button clicks.
showAllEmployees()	UI Event	Builds a TableView of all employees with ID, Name, Age, Department, Type, and Base Salary columns from AppLauncher.data.allEmployees.
showAddEmployee()	UI Event	Renders a scrollable form for all employee types; on Save constructs the correct Employee subclass, creates an EmployeeAccount, adds both to AppData, and calls DataStore.saveAll() to persist to CSV.
showCreateSubAdmin()	UI Event	Renders a username/password form; on Create calls SuperAdminAccount.promoteToSubAdmin() and adds the new SubAdminAccount to AppData — the new sub admin can log in immediately.
showLeaveRequests()	UI Event	Builds a TableView of leave requests with Approve and Decline buttons per row; both update LeaveStatus in place and refresh the table.
showSalaryOverview()	UI Event	Calls getOverallSalaryStats() and getTotalPaidByDept() from the PayrollManager and renders total, highest, lowest, average, and per-department figures.
runPayroll()	Core Payroll	Creates a new PayrollManager for the next month (preserving history), calls processMonthly(), updates all EmployeeAccount references, and shows a confirmation panel.

Method	Category	What It Does
showPayslipHistory()	UI Event	Builds a TableView of all payslips with Payslip ID, Employee Name, Month, Gross, and Net columns.
showDeptPayroll()	UI Event	Renders a table of Department to Total Net Paid using getTotalPaidByDept().
showAttendanceReport()	Attendance	Shows each employee's name, department, days recorded in the current month, and net salary in a TableView.
handleLogout()	Navigation	Loads roleselect.fxml and swaps the scene.
setContent(node)	Utility	Clears the StackPane contentArea and replaces it with the given Node — shared pattern across all three dashboard controllers.
buildTable()	Utility	Returns a styled dark-themed TableView with constrained column resize policy — shared helper called before adding columns.
col(title, property)	Utility	Creates a TableColumn with a PropertyValueFactory binding — used for simple field-to-column mappings.
fields(vbox)	Utility	Extracts all TextField children from a VBox, skipping Labels — used to read type-specific salary fields from the dynamic Add Employee form.
fmt(d)	Utility	Formats a double as a comma-separated 2-decimal-place string (e.g. 50,000.00).

SubAdminController — Frontend / Controller Layer

Method	Category	What It Does
initialize(url, rb)	JavaFX	No-op Initializable — content area populated on sidebar clicks.
showPayrollReport()	UI Event	Builds a TableView of all payslips with Payslip ID, Employee, Month, Gross, and Net from the payroll history.
showSalaryStats()	UI Event	Renders overall salary statistics (total, highest, lowest, average) and per-department totals.
showPayslipHistory()	UI Event	Displays the full payslip history table — same as Super Admin's version, scoped to Sub Admin access.
showLeaveRequests()	UI Event	Builds a leave request table with Approve and Decline buttons per row.
showAttendanceReport()	Attendance	Shows each employee's name, department, days recorded in the current month, and net salary.
showDeptPayroll()	UI Event	Renders a table of Department to Total Net Paid.
handleLogout()	Navigation	Clears AppLauncher.loggedInSubAdmin and loads roleselect.fxml.
setContent(node)	Utility	Replaces contentArea children with the given Node.
buildTable()	Utility	Returns a styled TableView — same helper pattern as SuperAdminController.
col(title, property)	Utility	Creates a PropertyValueFactory-bound TableColumn.
fmt(d)	Utility	Formats a double with commas and 2 decimal places.

EmployeeController — Frontend / Controller Layer

Method	Category	What It Does
initialize(url, rb)	JavaFX	Retrieves AppLauncher.loggedInEmployee, caches both the account and Employee reference, and sets the welcome label.
showPayslip()	UI Event	Filters the payroll history for payslips belonging to this employee and renders them in a TableView showing Payslip ID, Month, Gross, and Net.
showAttendance()	Attendance	Builds a TableView of AttendanceRecord with Date, Status, and Hours Worked columns.
showLeaveForm()	UI Event	Renders two DatePickers and a Submit button with full validation (no null dates, no past start, end must be after start); on valid submit calls EmployeeAccount.submitLeaveRequest() and adds to the global list.
showLeaveHistory()	Leave Mgmt	Displays the employee's personal leave request list in a table with ID, Start, End, and Status columns.
showMyReport()	Reporting	Shows the employee's name, department, role, gross/net salary, itemised allowances, and deductions.
showSalaryCalculator()	UI Event	Renders a calculator form with type-specific input fields pre-filled from the current employee's data; on Calculate shows a step-by-step breakdown without modifying any records.
generateReport()	Reporting	Renders a full employee summary: profile, salary, allowances, deductions, attendance summary (present/late/absent counts), and leave requests.
handleLogout()	Navigation	Clears AppLauncher.loggedInEmployee and loads roleselect.fxml.
setContent(node)	Utility	Replaces contentArea children with the given Node.
buildTable()	Utility	Returns a styled TableView — shared helper.
col(title, property)	Utility	Creates a PropertyValueFactory-bound TableColumn.
resultRow(label, value)	Utility	Creates an HBox row for the salary calculator breakdown display.
resultHighlight(l, v)	Utility	Creates a green HBox row for the final Net Salary line in the calculator breakdown.
parseOrZero(s)	Utility	Parses a string as a double, clamping to 0 on error or negative value — used by the salary calculator.
err(label, msg)	Utility	Sets a Label to red text with an error message — used in showLeaveForm() for date validation feedback.
fmt(d)	Utility	Formats a double with commas and 2 decimal places.

`Employee (abstract)` — Backend / Model Layer

Method	Category	What It Does
computeNetSalary()	Salary Calc	Calls calculateSalary() for the gross, sums all Deduction.getDeduction() values, returns gross minus total deductions — the shared net-pay formula via IPayable.
calculateSalary()	Interface	Abstract method — each concrete subclass defines its own gross salary formula, enabling polymorphic processing in PayrollManager.
addAttendanceRecord(r)	List Mgmt	Appends an AttendanceRecord to the internal mutable list.

Method	Category	What It Does
addLeaveRequest(l)	List Mgmt	Appends a LeaveRequest — called by EmployeeAccount.submitLeaveRequest() and during data initialisation.
addAllowance(a)	List Mgmt	Appends an Allowance — called in Main.addSeedAllowancesDeductions() for full-time employees.
addDeduction(d)	List Mgmt	Appends a Deduction — called in Main and iterated by computeNetSalary().
getDetails()	Utility	Returns a multi-line string of all six core fields — used in manageAllEmployees() and generateOwnReport().
getAttendanceRecords()	Getter	Returns an unmodifiable view of the internal attendance list — prevents external mutation.
getLeaveRequests()	Getter	Returns an unmodifiable view of the leave requests list.
getAllowances()	Getter	Returns an unmodifiable view of the allowances list.
getDeductions()	Getter	Returns an unmodifiable view of the deductions list.

FullTimeEmployee — Backend / Model Layer

Method	Category	What It Does
calculateSalary()	Salary Calc	Returns baseSalary + allowances + (2000 x overtimeHrs) — the gross formula for full-time staff. Both fields are validated >= 0 in the constructor.
getTotalAllowances()	Getter	Returns the flat allowances amount — used by DataStore.saveAll() for CSV serialisation and by the salary calculator.
getOvertimeHrs()	Getter	Returns the overtime hours count — used in CSV serialisation and the salary calculator.

PartTimeEmployee — Backend / Model Layer

Method	Category	What It Does
calculateSalary()	Salary Calc	Returns hourlyRate x hoursWorked — simple hourly gross. Constructor validates hourlyRate > 0 and hoursWorked >= 0.
getHourlyRate()	Getter	Returns the hourly rate — used in CSV serialisation and the salary calculator.
getHoursWorked()	Getter	Returns the hours worked count — used in CSV serialisation.

ContractEmployee — Backend / Model Layer

Method	Category	What It Does
calculateSalary()	Salary Calc	Returns contractRate directly — flat monthly pay. Constructor validates contractRate > 0 and contractEnd != null.
getContractRate()	Getter	Returns the monthly contract rate — used in CSV serialisation and the salary calculator.
getContractEnd()	Getter	Returns the contract end date (LocalDate) — displayed in employee tables.

`PayrollManager` — Backend / Model Layer

Method	Category	What It Does
processMonthly()	Core Payroll	Iterates all employees, calls calculateSalary() and computeNetSalary(), creates a PaySlip with auto-incremented ID (PS-YYYY-MM-N), and appends to payrollHistory.
getPayslipByEmployee(emp, targetMonth)	Query	Searches payrollHistory for a PaySlip matching the employee reference and YearMonth; returns null if not found.
getPayslipById(payslipId)	Query	Searches payrollHistory by String ID — available for future lookup features.
getTotalPaidByDept()	Analytics	Builds a LinkedHashMap of department to total net salary using Map.merge.
getOverallSalaryStats()	Analytics	Returns a map with total, highest, lowest, and average net salaries.
getPayrollHistory()	Getter	Returns the full ArrayList of PaySlips.
getMonth()	Getter	Returns the YearMonth this manager was created for.
getEmployees()	Getter	Returns the employee list this manager operates on.

`PaySlip` — Backend / Model Layer

Method	Category	What It Does
generate()	Core Payroll	Recomputes grossSalary and netSalary from the linked employee — allows refreshing a slip if salary data changes after initial creation.
reGenerate()	Core Payroll	Alias for generate() — provided for semantic clarity when re-running salary calculations.
getPaySlipId()	Getter	Returns the unique payslip ID string (format: PS-YYYY-MM-N).
getMonth()	Getter	Returns the YearMonth this payslip was generated for.
getGrossSalary()	Getter	Returns the gross salary stored in this immutable snapshot.
getNetSalary()	Getter	Returns the net salary stored in this immutable snapshot.
getEmployee()	Getter	Returns the linked Employee reference.

`LeaveRequest` — Backend / Model Layer

Method	Category	What It Does
setLeaveStatus(leaveStatus)	Leave Mgmt	Updates the leave status to APPROVED or REJECTED — called by Approve/Decline buttons in all admin controllers.
isNearingEnd()	Utility	Returns true if today is within 2 days of the end date — can be used to display leave-expiry warnings.
getDaysRemaining()	Utility	Returns the number of days from today until the end date using ChronoUnit.DAYS.
getRequestId()	Getter	Returns the integer request ID.
getStartDate()	Getter	Returns the leave start date.

Method	Category	What It Does
getEndDate()	Getter	Returns the leave end date.
getEmployee()	Getter	Returns the Employee who submitted this request.
getLeaveStatus()	Getter	Returns the current LeaveStatus (PENDING / APPROVED / REJECTED).

`AttendanceRecord` — Backend / Model Layer

Method	Category	What It Does
getHourWorked()	Getter	Returns the hours worked for this record.
setHourWorked(hourWorked)	Setter	Updates the hours worked.
getAttendanceStatus()	Getter	Returns the AttendanceStatus (PRESENT / LATE / ABSENT / LEAVE).
setAttendanceStatus(s)	Setter	Updates the attendance status.
getDate()	Getter	Returns the LocalDate of this attendance record.
setDate(date)	Setter	Updates the record date.

`Allowance` — Backend / Model Layer

Method	Category	What It Does
compute()	Salary Calc	Returns the allowance amount — semantic alias for getAmount(), designed for future percentage-based extension.
getAmount()	Getter	Returns the raw allowance amount — iterated via the Allowance list.
getAllowanceType()	Getter	Returns the AllowanceType (HOUSING / TRANSPORT / MEDICAL / BONUS).

`Deduction` — Backend / Model Layer

Method	Category	What It Does
calculateDeduction()	Salary Calc	Returns the deduction amount — iterated by Employee.computeNetSalary() to sum total deductions.
getDeduction()	Getter	Returns the raw deduction amount used directly in the net salary formula.
getDeductionType()	Getter	Returns the DeductionType (TAX / PENSION / INSURANCE / FINES).

`ReportEngine` — Backend / Reporting Layer

Method	Category	What It Does
salarySummaryReport()	Reporting	Prints Name, Gross, and Net for every PaySlip in history — called by SubAdminAccount.viewReports().
deptPayrollReport()	Reporting	Prints each department and its total net salary.

Method	Category	What It Does
attendanceSalaryReport(month)	Reporting	For each employee, counts attendance records in the given month and prints them alongside net salary.
payslipHistoryReport()	Reporting	Prints full payslip history: ID, employee name, month, and net salary.
getTotalPaidReport()	Reporting	Prints department to total paid — similar to deptPayrollReport() with a different header label.

9. Sample Dataset — 28 Employees

ID	Name	Type	Dept	Base/Rate	Extra
101	Ali Hassan	**Full-Time**	IT	50,000	OT: 10h
102	Sara Khan	**Full-Time**	HR	55,000	OT: 5h
103	Bilal Ahmed	**Full-Time**	Finance	60,000	OT: 8h
104	Ayesha Malik	**Full-Time**	IT	52,000	OT: 12h
105	Usman Tariq	**Full-Time**	Finance	65,000	OT: 6h
106	Fatima Zahra	**Full-Time**	HR	53,000	OT: 4h
107	Hamza Raza	**Full-Time**	IT	58,000	OT: 9h
108	Zainab Ali	**Full-Time**	Finance	62,000	OT: 7h
109	Omar Sheikh	**Full-Time**	HR	70,000	OT: 3h
110	Nadia Hussain	**Full-Time**	IT	48,000	OT: 11h
111	Kamran Baig	**Part-Time**	IT	500/hr	80h/mo
112	Sana Mirza	**Part-Time**	HR	450/hr	70h/mo
113	Tariq Mehmood	**Part-Time**	Finance	550/hr	90h/mo
114	Rabia Nawaz	**Part-Time**	IT	480/hr	75h/mo
115	Fahad Iqbal	**Part-Time**	HR	520/hr	85h/mo
116	Hina Qureshi	**Part-Time**	Finance	460/hr	65h/mo
117	Asad Javed	**Part-Time**	IT	510/hr	88h/mo
118	Mariam Farooq	**Part-Time**	HR	490/hr	72h/mo
119	Rizwan Siddiqui	**Part-Time**	Finance	530/hr	92h/mo
120	Amna Butt	**Part-Time**	IT	470/hr	68h/mo
121	Shahid Afridi	**Contract**	IT	80,000/mo	End: Mar 2027
122	Mehwish Hayat	**Contract**	HR	75,000/mo	End: Feb 2027
123	Faisal Qureshi	**Contract**	Finance	90,000/mo	End: Jan 2027
124	Mahnoor Baloch	**Contract**	IT	70,000/mo	End: Dec 2026
125	Danish Taimoor	**Contract**	HR	85,000/mo	End: Nov 2026
126	Aiman Zaman	**Contract**	Finance	72,000/mo	End: Oct 2026
127	Wahaj Ali	**Contract**	IT	78,000/mo	End: Sep 2026
128	Yumna Zaidi	**Contract**	HR	82,000/mo	End: Aug 2026

10. System Screenshots

The following screenshots demonstrate the running system across all three role-based interfaces.

10.1 Role Selection Screen

First screen on launch. The user selects Admin or Employee, which routes to the appropriate login flow.

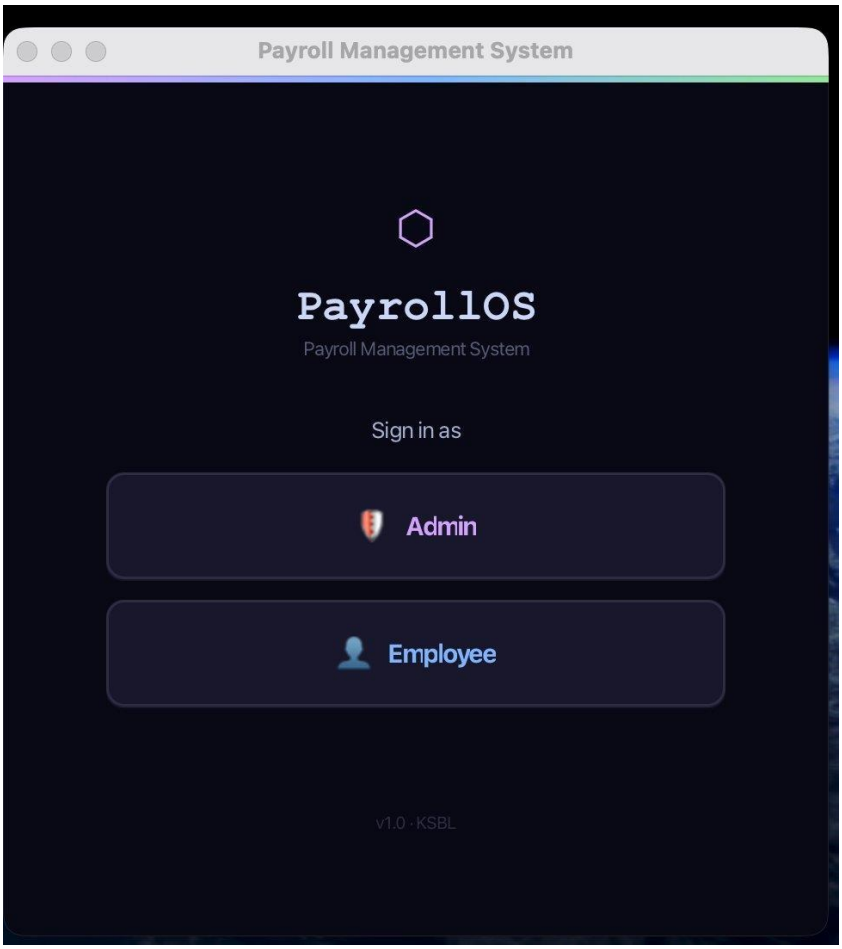

Figure 1 — PayrollOS Role Select Screen (roleselect.fxm

10.2 Super Admin Panel — All Employees

Super Admin dashboard showing the full employee table with ID, Name, Age, Department, Type, and Base Salary. All 28 seed employees are loaded from CSV on startup. Part-Time employees show Base Salary as 0.0 (their salary is calculated hourlyRate x hoursWorked, not baseSalary).

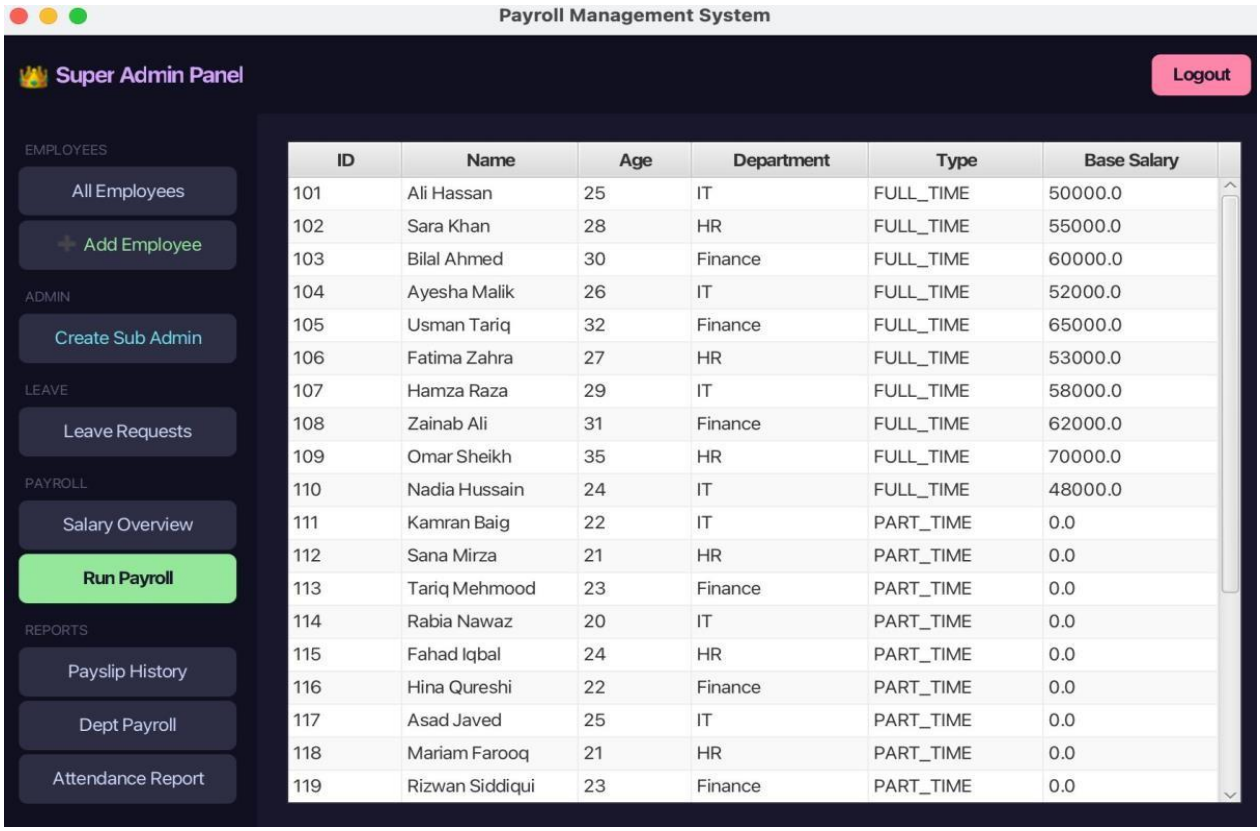

Figure 3 — Super Admin Panel: All Employees TableView (superadmin.fxml / SuperAdminController)

10.3 Super Admin Panel — Add New Employee

The Add New Employee form lets the Super Admin register a new staff member by entering their full name, age, department (IT / HR / Finance), login credentials, and employee type (Full-Time, Part-Time, or Contract). On save, the new record is persisted to employees.csv and immediately appears in the All Employees table.

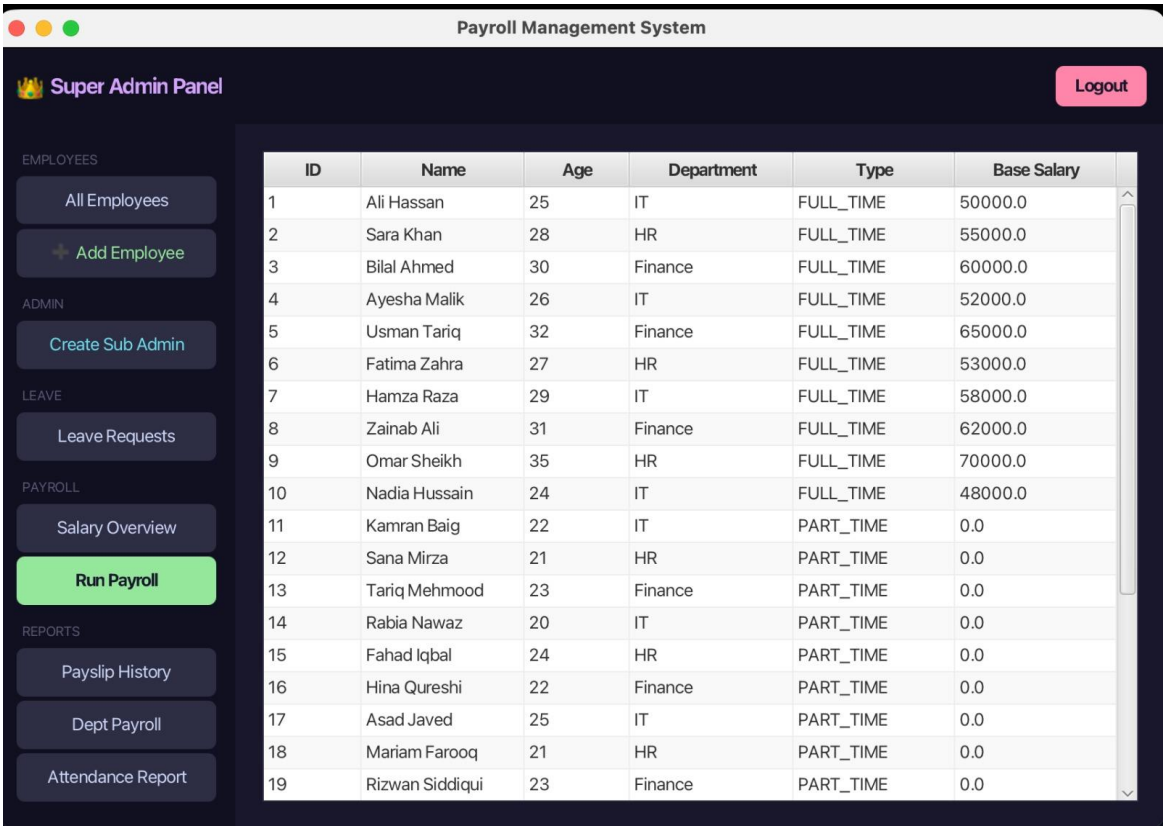

Figure 4 — Super Admin Panel: Add New Employee Form (superadmin.fxml / SuperAdminController)

10.4 Super Admin Panel — Create Sub Admin

The Create Sub Admin Account screen allows the Super Admin to provision a new Sub Admin at runtime by entering a username and password. Internally this calls SuperAdminAccount.promoteToSubAdmin(), which instantiates a new SubAdminAccount and injects it into the live session without requiring a restart.

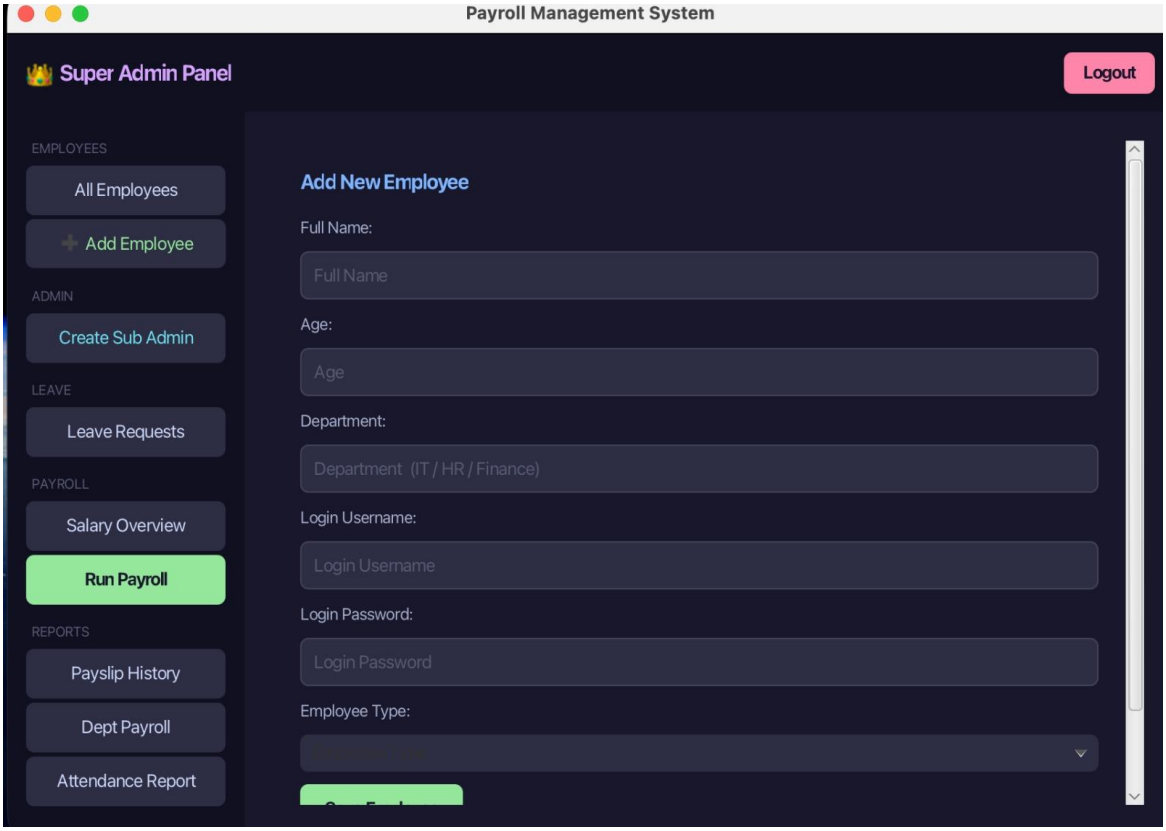

Figure 5 — Super Admin Panel: Create Sub Admin Account (superadmin.fxml / SuperAdminController)

10.5 Run Payroll — Payroll Processed Confirmation

After clicking Run Payroll, PayrollManager.processMonthly() iterates all 28 employees, computes gross and net salaries, and generates a uniquely-IDed PaySlip per employee. The confirmation screen displays the processed month (e.g. 2026-06) and the total payslip count. The user can then navigate to Payslip History to review individual payslips.

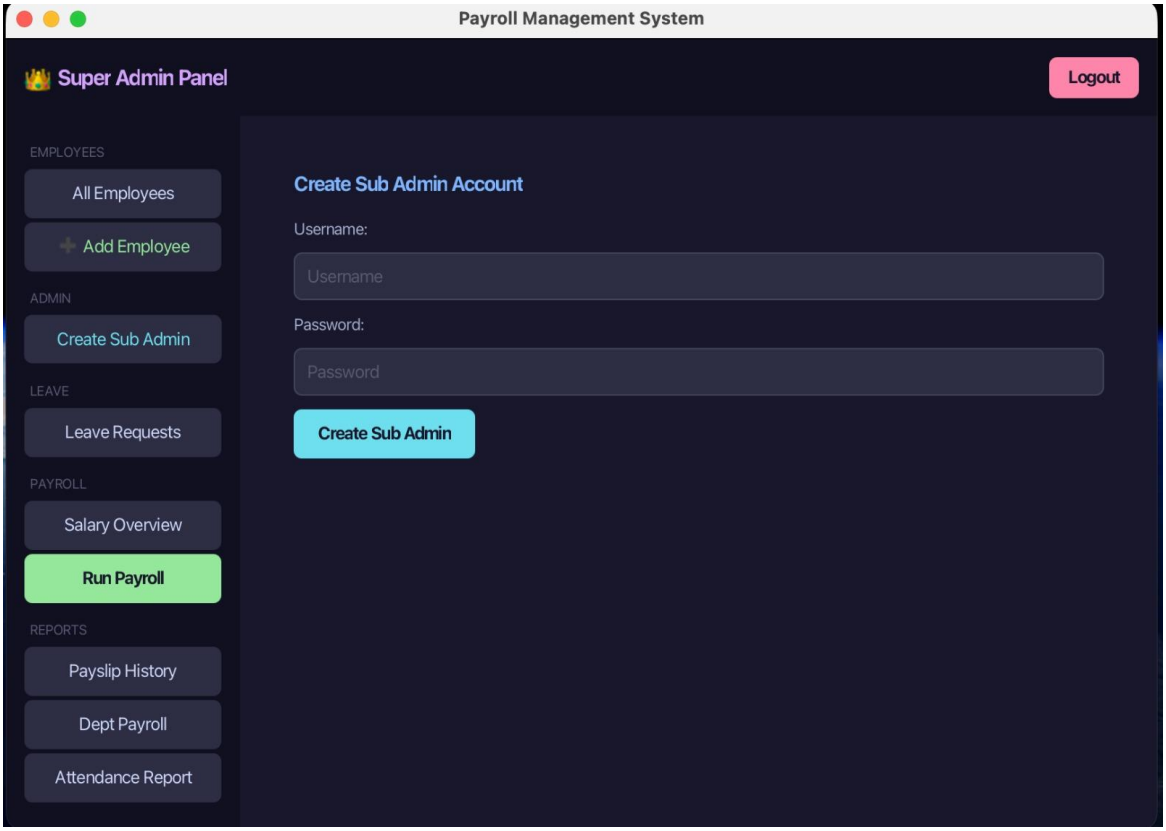

Figure 6 — Run Payroll: Payroll Processed Confirmation (superadmin.fxml / SuperAdminController)

Sub Admin Panel — Payroll Report

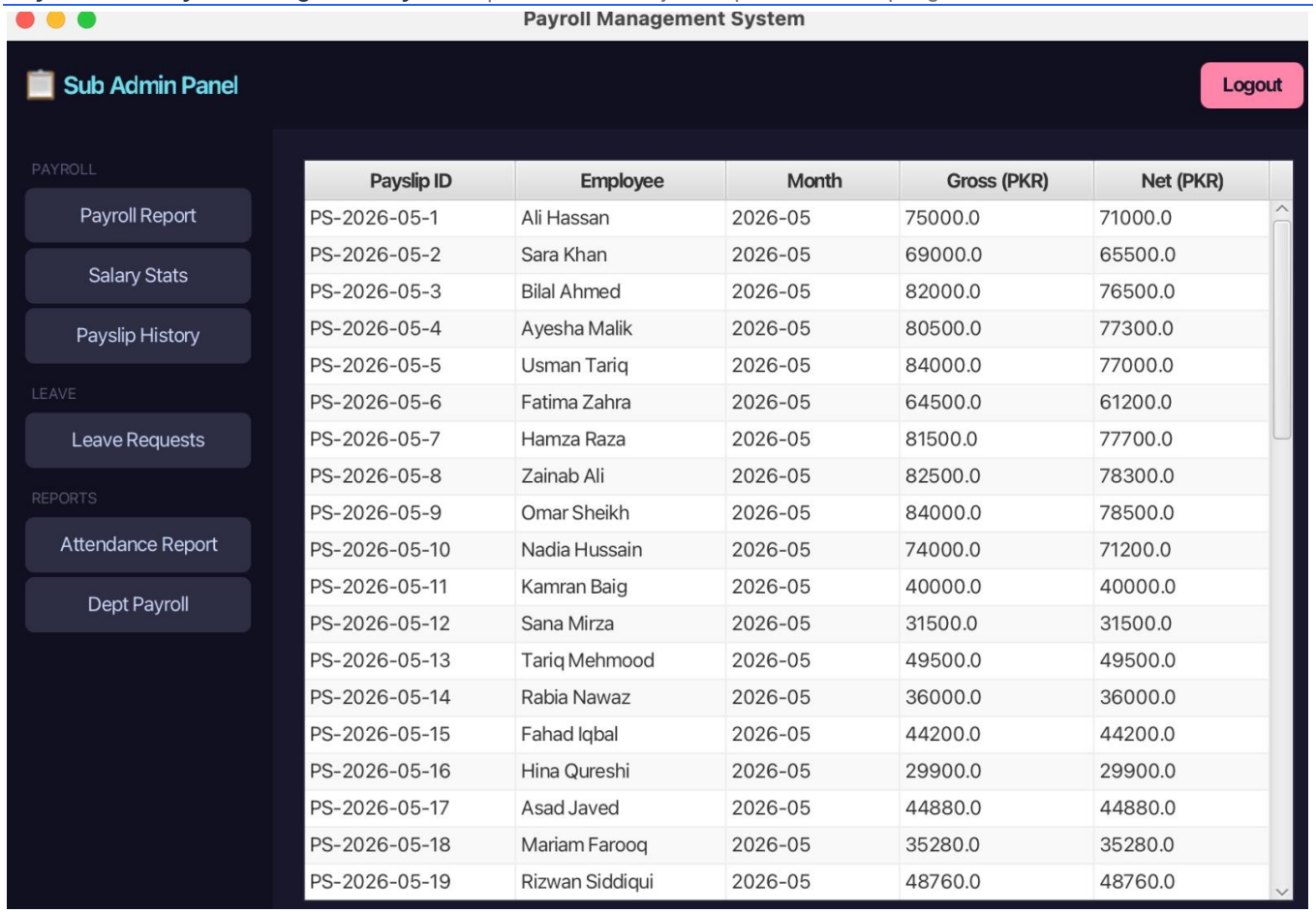

Figure 7 — Sub Admin Panel: Payroll Report (subadmin.fxml / SubAdminController)

Sub Admin Panel — Leave Requests

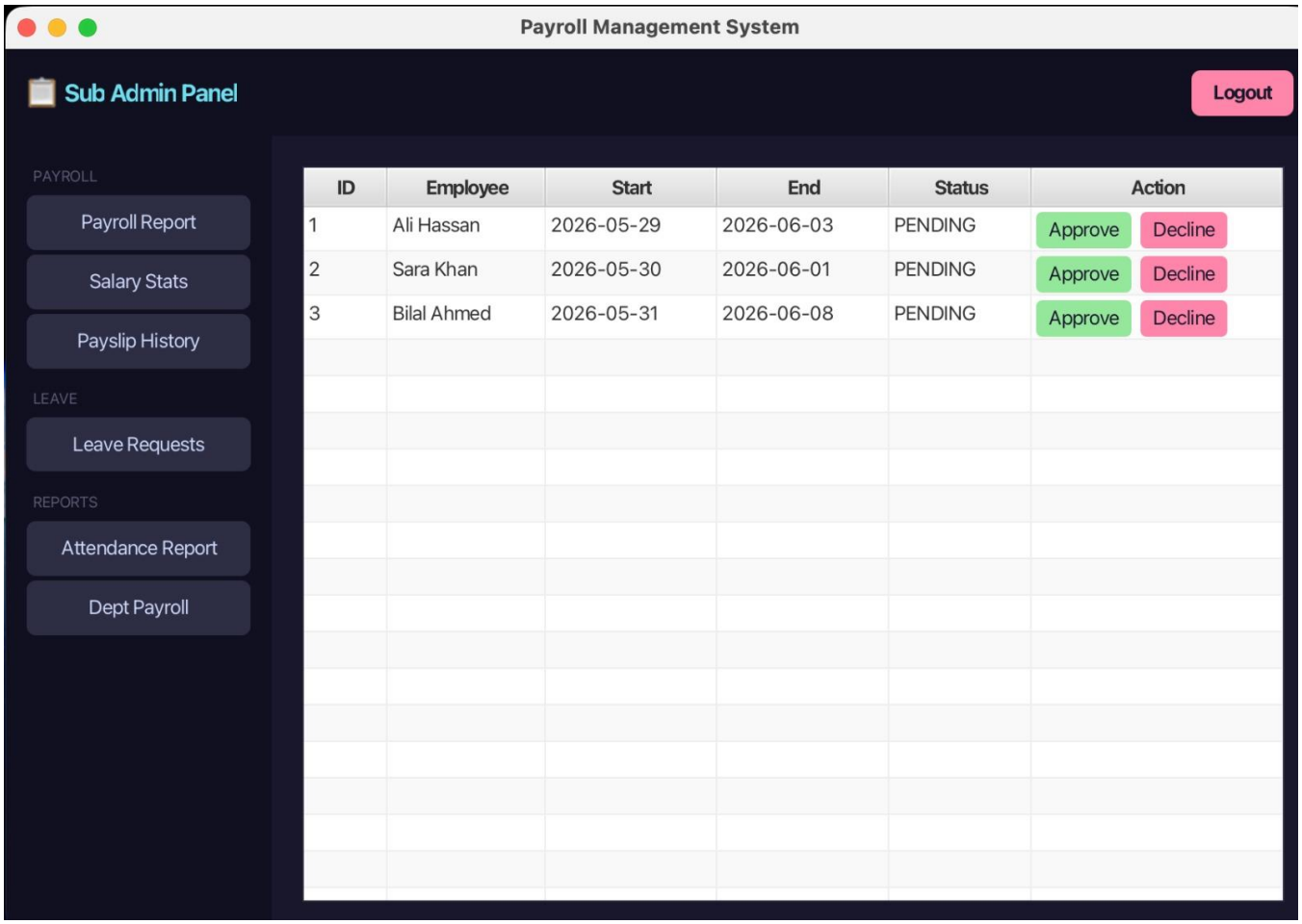

Figure 8 — Sub Admin Panel: Leave Requests (subadmin.fxml / SubAdminController)

Employee Portal — My Payslips

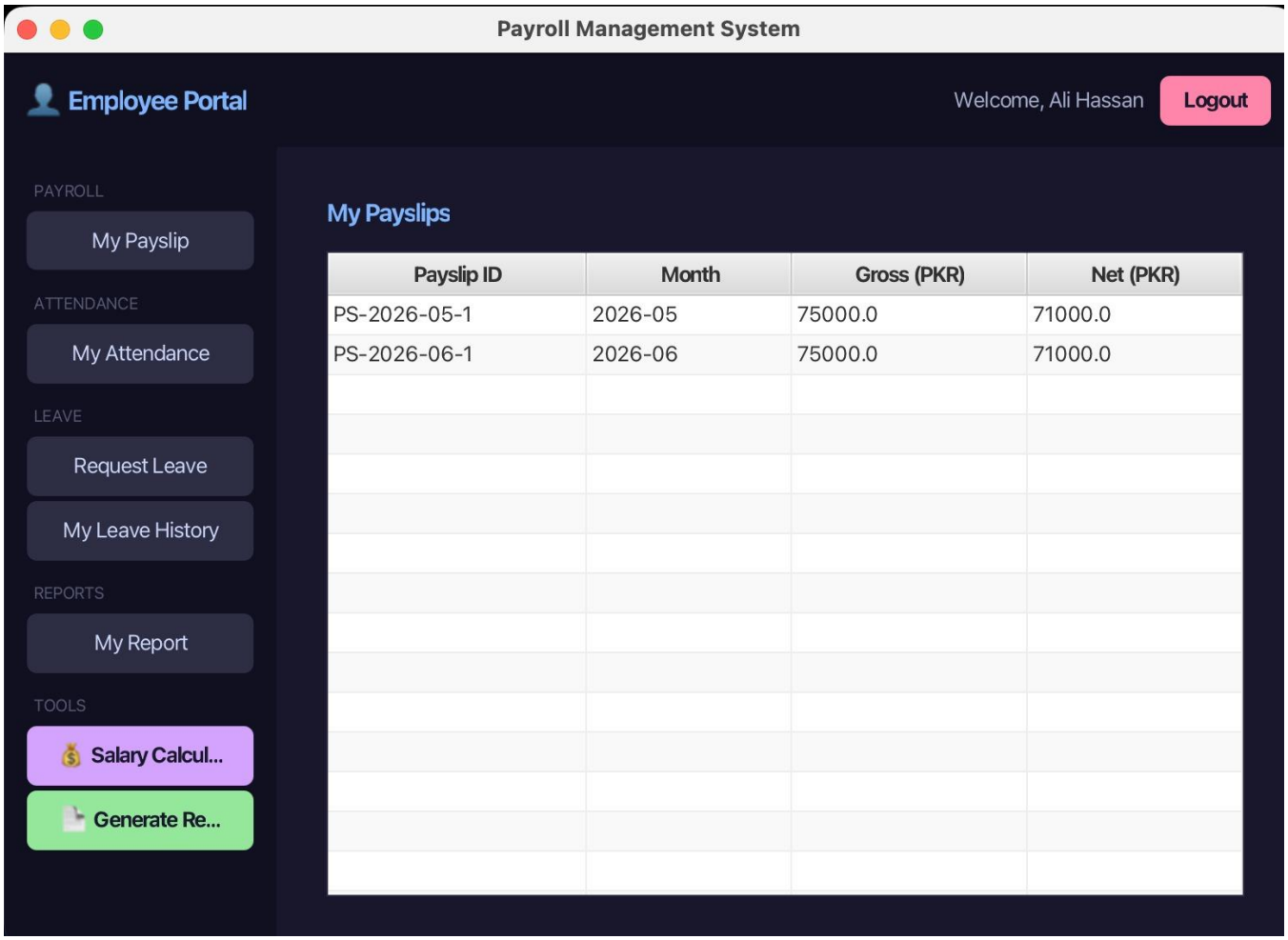

Figure 9 — Employee Portal: My Payslips (employee.fxml / EmployeeController)

Employee Portal — Submit Leave Request

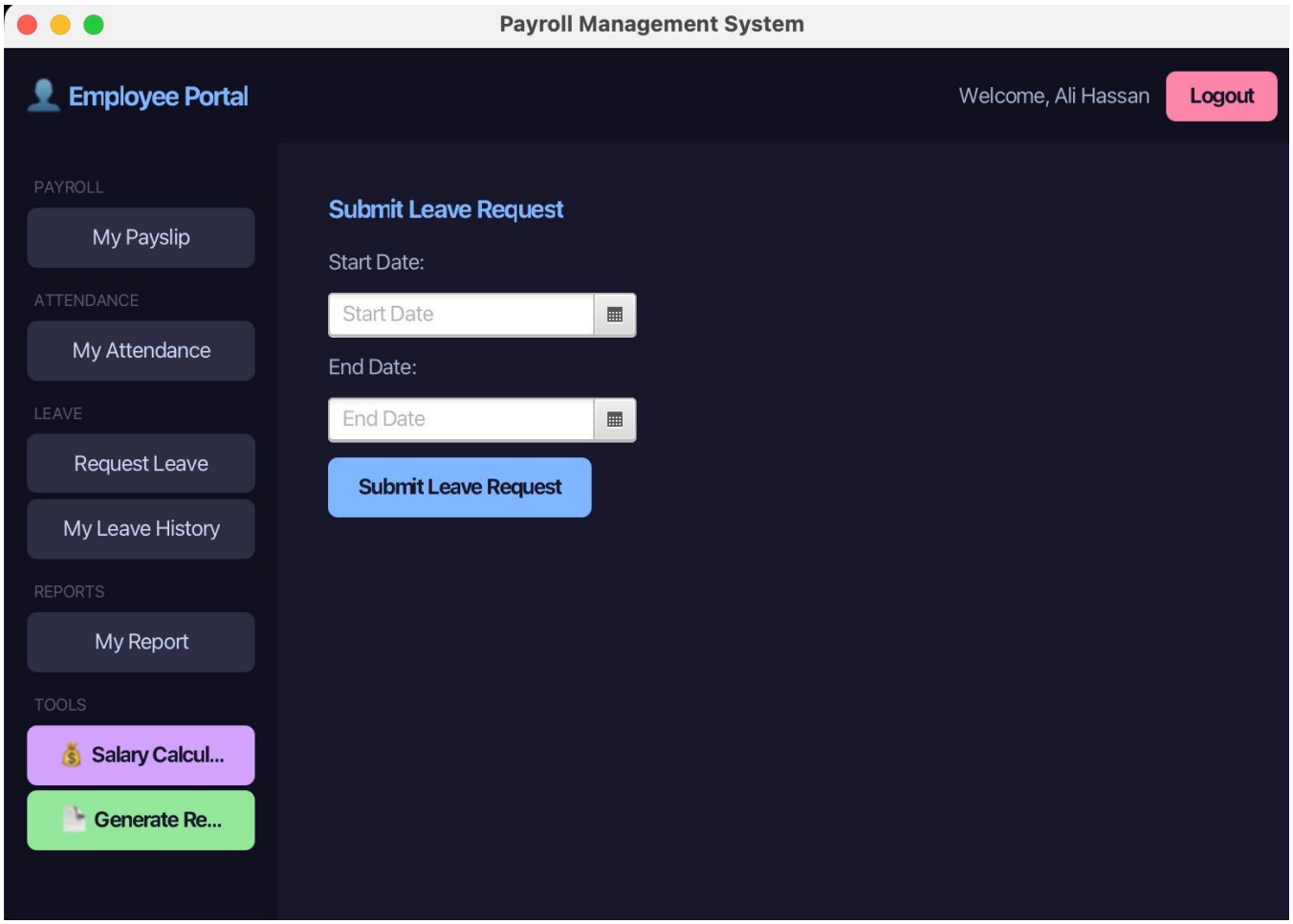

Figure 10 — Employee Portal: Submit Leave Request (employee.fxml / EmployeeController)

Employee Portal — Salary Calculator

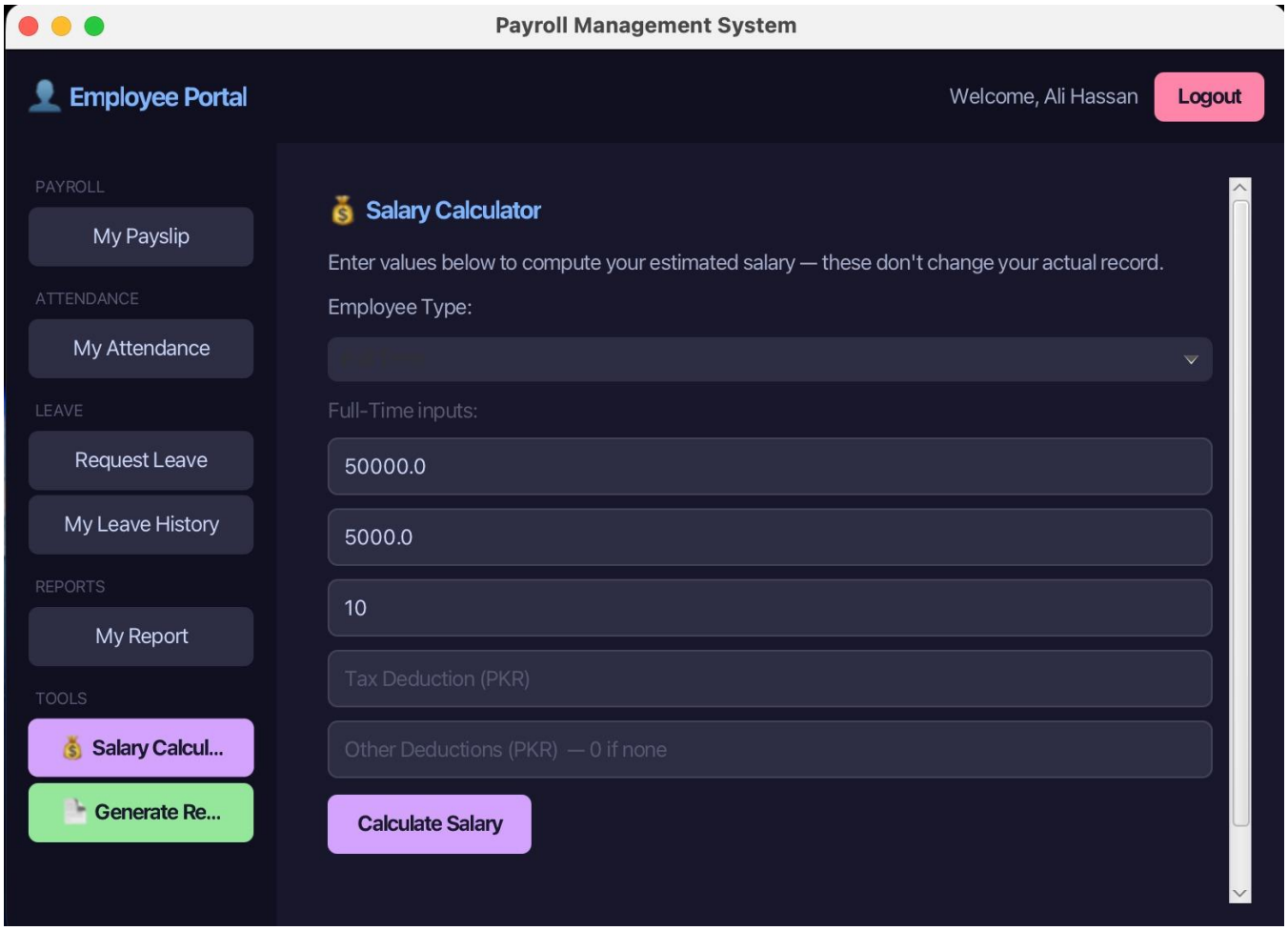

Figure 11 — Employee Portal: Salary Calculator (employee.fxml / EmployeeController)

11. Conclusion

PayrollOS represents a production-grade desktop payroll application built entirely in Java. The multi-screen navigation flow, CSV persistence, dynamic Sub Admin creation, interactive salary calculator, and the expanded Employee dashboard together form a coherent, deployable system.

The codebase demonstrates strong OOP principles throughout: polymorphic salary calculation across three employee types, clean MVC separation between FXML views and Java model classes, defensive encapsulation of internal lists via unmodifiable views, Java records for lightweight data transfer, and pattern-matching instanceof for type-safe CSV serialisation.

With the addition of password hashing and persistent leave/attendance storage, the system would be ready for real-world deployment in a small to mid-sized organisation.